Database Migrations with Flyway

How and why each migration step was done in the DBMigCrud project, and how it compares with letting Hibernate manage the schema

Spring Boot 4.1.2 | Java 21 | PostgreSQL 18.1 | Spring Data JPA | Flyway | 12 November 2026

The short version

- With Hibernate `ddl-auto=update` you change the entity and Hibernate works out the SQL. With Flyway you write the SQL yourself in numbered files, and Flyway runs each file exactly once.
- In this project Flyway owns the schema. Hibernate is set to `validate`: it compares the entities with the tables and changes nothing.
- The schema was built in five migrations, V1 to V5. V1 was the starting point and V2 to V5 were the four learning steps.
- The cost of Flyway is that every change is made twice, once in SQL and once in Java. What you get back is a recorded history, the same schema in every database, and control over existing data.
- From step 2 onward, each step needed something that Hibernate's automatic update cannot do or does unreliably: values for existing rows, a rule on an existing column, seed data, and an index.

1. Two ways to manage a database schema

Hibernate schema generation

Hibernate can read your `@Entity` classes and create or change the tables for you. The property `spring.jpa.hibernate.ddl-auto` decides how far it goes.

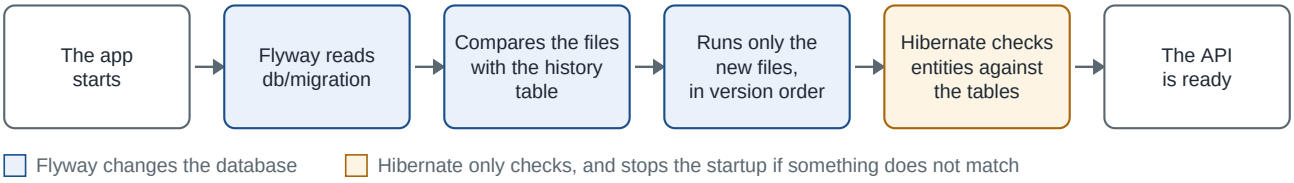
Value	What Hibernate does at startup
`none`	Nothing.
`validate`	Checks that the tables and columns the entities need exist with matching types. Stops the startup if they do not. Changes nothing. **This project uses this value.**
`update`	Adds tables and columns that are missing. It does not drop or rename anything.
`create`	Drops the tables and creates them again. All data is lost on every start.
`create-drop`	Like `create`, and it also drops the tables when the app stops.

Flyway

Flyway does not look at your Java code at all. It runs SQL files that you write. Each file is one **migration**: a small, numbered change to the database.

- **The file name carries the version.** In `V2__add_price_to_products.sql`, `V` means a versioned migration, `2` is the version, the double underscore is the separator, and the rest is the description.
- **Flyway remembers what it has run.** It keeps a table called `flyway_schema_history` with one row per applied migration: version, description, checksum, time and success.
- **Only new files run.** At startup Flyway compares the files with the history table and runs the ones that are not there yet, in version order.
- **Applied files are frozen.** The checksum is a fingerprint of the file. If you edit a file after it has run, the fingerprint no longer matches and Flyway refuses to start.

What happens when this project starts



2. How this project is set up

- **Dependencies** in `pom.xml`: `spring-boot-starter-flyway` and `flyway-database-postgresql`. Spring Boot runs Flyway automatically when they are present.
- **Migration folder:** `src/main/resources/db/migration`. This is the default place Flyway looks.
- **Credentials** come from the `.env` file through placeholders in `application.properties`. The `.env` file is not committed to git.

```
spring.config.import=optional:file:.env[.properties]

spring.datasource.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

# Flyway owns the schema. Hibernate only checks that the entities match it.
spring.jpa.hibernate.ddl-auto=validate
```

The routine for every change

1. Write the next SQL file, for example `V6__describe_the_change.sql`.
2. Change the entity class by hand so it matches the new table structure.
3. Change the service if the new field should be saved or updated through the API.
4. Restart the app. Flyway applies the new file, then Hibernate validates the entities.

Flyway never edits Java code, and Hibernate never edits the database. The `validate` setting is the only link between them: it checks your work but does not fix it.

3. The migrations, step by step

Version	Step	Change	Main lesson
V1	Start	Create `products` with `id` and `name`	Flyway creates the first table and its own history table.
V2	Step 1	Add a nullable `price` column	The simplest migration. Only new files run.
V3	Step 2	Add a `NOT NULL` column `stock`	Rows that already exist need a value.
V4	Step 3	Make `name` unique	A constraint depends on the data already in the table.
V5	Step 4	Add `categories` and link `products` to it	Several statements in one file, including data.

About the Hibernate comparisons below: they describe how `ddl-auto=update` normally behaves. This project never ran in that mode, so those parts are expected behaviour, not something that was tested here. Everything under "What we saw" was observed on the real database.

Starting point (V1): create the first table

`V1__create_products_table.sql`

```
CREATE TABLE products (
    id   BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);
```

How we did it. The `Product` entity started with only `id` and `name`, so there was room to grow it in later steps. The SQL file creates exactly that table.

Why a migration. Even the first table is created by a migration. That way an empty database can always be brought to the current schema by running the files in order, with nothing done by hand.

> **With Hibernate instead.** `create` or `update` would have created the table from the entity with no SQL file to write. For the first day of a project that is faster. The difference is that the SQL is never stored anywhere, and the column types are Hibernate's defaults rather than your decision.

What we saw. On the first start Flyway found an empty database, created its history table and applied V1:

```
Schema history table "public"."flyway_schema_history" does not exist yet
Creating Schema History table "public"."flyway_schema_history" ...
Migrating schema "public" to version "1 - create products table"
Successfully applied 1 migration to schema "public", now at version v1
```

Step 1 (V2): add a nullable column

`V2__add_price_to_products.sql`

```
ALTER TABLE products
    ADD COLUMN price NUMERIC(10, 2);
```

How we did it. One SQL statement adds the column. In Java we added a `BigDecimal price` field with a getter and setter to `Product`, and one line in `ProductService.update` so the PUT endpoint also saves the price.

Why a migration. The column is nullable, so the row that already existed needed no value. This is the easiest kind of change and a good first step to see the file naming and the history table at work.

> **With Hibernate instead.** `update` would have added this column automatically from the new field. For a plain nullable column Hibernate is less work and the result in the database is the same. What Flyway adds is the record: this change is version 2, it has a description and a date, and every other database will receive exactly the same statement.

What we saw. Flyway saw the database was at version 1 and ran only V2. The existing Keyboard row came back with `"price":null`. Creating a product with a price returned 201.

```
Successfully validated 2 migrations
Current version of schema "public": 1
Migrating schema "public" to version "2 - add price to products"
Successfully applied 1 migration to schema "public", now at version v2
```

Step 2 (V3): add a NOT NULL column to a table that has rows

`V3__add_stock_to_products.sql`

```
ALTER TABLE products
    ADD COLUMN stock INTEGER NOT NULL DEFAULT 0;
```

How we did it. The statement adds the column with a default. In Java we added `private Integer stock = 0;` with `@NotNull` and `@PositiveOrZero`, and the service update now copies `stock` too. The `= 0` gives the same default on the Java side when a request leaves the field out.

Why a migration. The table already had two rows. A `NOT NULL` column needs a value in every row the moment it appears. Without `DEFAULT 0` PostgreSQL rejects the statement because the old rows would contain nulls. With it, PostgreSQL fills the old rows with 0 and also uses 0 for later inserts that omit the column.

> **With Hibernate instead.** For a field marked `nullable = false`, `update` generates `ADD COLUMN ... NOT NULL` with no default, which PostgreSQL rejects on a table that has rows. By default Hibernate only logs that error and keeps starting, so the app would run without the column and fail later at the first query. You can work around it by putting SQL fragments into annotations (`columnDefinition` or `@ColumnDefault`), but Hibernate has no way to run a backfill such as an `UPDATE` for you.

What we saw. Both existing rows showed `"stock":0` after the migration. A new product sent without `stock` was stored with 0. Sending `"stock":null` or a negative number returned 400.

Step 3 (V4): add a rule to a column that already exists

`V4__add_unique_name_to_products.sql`

```
ALTER TABLE products
    ADD CONSTRAINT uk_products_name UNIQUE (name);
```

How we did it. This migration adds no column. It adds a named unique constraint on `name`. In Java we set `unique = true` on the field, which only documents the rule, and added `ProductExceptionHandler` so a duplicate name returns 409 Conflict and not a 500 error.

Why a migration. A constraint depends on the data, not only on the structure. It applied cleanly because the four products had different names. If two rows had shared a name, PostgreSQL would have rejected it, and the migration would first have needed a statement to clean up the duplicates. Giving the constraint a name (`uk_products_name`) means a later migration can drop or change it by name, and the name appears in error messages.

> **With Hibernate instead.** `update` is built to add missing tables and columns. It does not reliably change a column that already exists. Adding `unique = true` to an existing field may be applied, skipped, or only logged as a warning, depending on the Hibernate version and settings, and the constraint gets a generated name. Also note that `validate` does not check unique constraints, so with Flyway the SQL file is the one place where this rule is really defined.

What we saw. Creating a second product named Keyboard returned 409. Renaming Monitor to Keyboard returned 409. Updating Keyboard while keeping its own name returned 200. The rejected insert still used up an id, so the next product got id 7 and not 6. Gaps like this are normal for identity columns.

Step 4 (V5): add a second table and link the two

`V5__create_categories_and_link_products.sql`

```
CREATE TABLE categories (
    id   BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    CONSTRAINT uk_categories_name UNIQUE (name)
);

INSERT INTO categories (name)
VALUES ('Peripherals'),
       ('Displays'),
       ('Audio');

ALTER TABLE products
    ADD COLUMN category_id BIGINT,
    ADD CONSTRAINT fk_products_category FOREIGN KEY (category_id) REFERENCES categories (id);

CREATE INDEX idx_products_category_id ON products (category_id);
```

How we did it. One file with four statements. In Java we added a `Category` entity with its repository and a read-only `GET /api/categories` endpoint, a `@ManyToOne` field `category` on `Product`, and a lookup in `ProductService` that returns 400 when the category id in the request does not exist.

Why a migration.

- **Order matters inside the file.** `categories` must exist before `products` can reference it.
- **All or nothing.** On PostgreSQL Flyway runs the file in one transaction. If any statement fails, none of it is applied.
- **Data travels with the schema.** The three categories are inserted by the migration, so every database that runs V5 gets the same rows.
- **The foreign key is nullable on purpose.** The products that already existed had no category, so a `NOT NULL` foreign key would have failed for the same reason as in step 2.
- **The index is a separate decision.** PostgreSQL does not index a foreign key column automatically.

> **With Hibernate instead.** `update` would have created the `categories` table, the `category_id` column and the foreign key from the `@ManyToOne` mapping. It would not have inserted the three category rows; those need a separate mechanism such as Spring's `data.sql`, which runs on each start instead of once and so must be safe to repeat. It would not have created the index unless it was declared with `@Index`, and the foreign key would get a generated, unreadable name unless one was set in an annotation.

What we saw. `GET /api/categories` returned the three seeded rows. Existing products showed `"category":null`. Creating a product with `"category":{"id":3}` returned 201 with the full category in the response. An unknown category id returned 400.

4. Flyway compared with Hibernate

The table compares Flyway with `ddl-auto=update`, the Hibernate mode that also tries to keep existing data.

	Hibernate ddl-auto=update	Flyway with ddl-auto=validate
Who writes the SQL	Hibernate, from the entity classes.	You, by hand.
Work per change	One edit: the entity.	Two edits that must agree: the SQL file and the entity.
Record of changes	None. The database only shows its current state.	The history table lists every change with version, description, time and result.
Same schema in every database	Not guaranteed. The result depends on what each database looked like before.	Yes. Every database runs the same files in the same order.
Existing data	Not considered. It cannot fill in or convert values.	Full control: `DEFAULT`, `UPDATE` and `INSERT` inside the migration.
Rename or remove	It does not drop or rename. A renamed field becomes a new empty column, and the old column stays with the data.	Whatever SQL you write, for example `RENAME COLUMN` or `DROP COLUMN`.
Constraints and indexes	Only what annotations declare. Changes to existing columns are unreliable. Names are generated.	Anything the database supports, with names you choose.
When a change fails	By default the error is logged and the app keeps starting.	The startup stops. On PostgreSQL the whole file is rolled back and the database stays at the previous version.
Code review	Reviewers see the entity change, not the SQL that will run.	The exact SQL is in the commit.
Changing database product	Easier. Hibernate adapts the SQL to each database.	Harder. The files are written for one database, here PostgreSQL.
Going back	No undo.	No automatic undo in the free edition. You fix a mistake with a new migration.
Good fit	Learning JPA, prototypes, demos, throwaway and test databases.	Any database whose data you want to keep, shared databases, production.

5. The trade-off

What Flyway cost us in this project

- **Every change was made twice.** Each step needed a SQL file and a matching edit to `Product.java`. If you write the entity field but forget the migration, the app fails at startup with a schema-validation error. If you write the migration but forget the field, the app starts and the column exists, but the API cannot see it.
- `validate` **checks less than you might expect.** It checks tables, columns and types. It does not check unique constraints, indexes or defaults. The rule from step 3 exists only in the SQL.
- **Applied migrations cannot be edited.** A typo in V3 is not fixed in V3; it is fixed in a new file.
- **You need to know the SQL of your database.** Hibernate hides it; Flyway does not.
- **Teams need a numbering rule.** Two people can both create `V6` on different branches. Some teams use timestamps as version numbers to avoid this.

What it gave us

- **Control over existing rows.** Step 2 filled old rows with a default, and step 4 kept old products valid with a nullable foreign key. Both were explicit decisions written in SQL.
- **Things Hibernate's update does not do well.** A constraint on an existing column, seed rows, a chosen constraint name and an index were each one line of SQL.
- **A database that can be rebuilt.** Point the app at an empty PostgreSQL database and start it; Flyway runs V1 to V5 and the schema is identical.
- **A readable history.** You can see what changed, in which order, and when.
- **Early failure.** If the entities and the schema disagree, the app does not start, so you find out at startup and not from a failing request.

Which one to use

- Use Hibernate `create-drop` or `update` while learning JPA, for a quick prototype, or for tests against a throwaway database.
- Use Flyway with `validate` once the database holds data you care about, or as soon as there is more than one database (for example your laptop and a server).
- Do not use `update` or `create` on a production database.
- A useful habit: let Hibernate print the SQL it would generate, use that as a first draft, and then edit it into a migration file.

6. Quick reference

Rules to remember

1. Never edit or delete a migration that has been applied. Add a new one.
2. A new file needs a higher version number than every file already applied.
3. The SQL file and the entity change together, in the same commit.
4. Before adding `NOT NULL` or a constraint, think about the rows that already exist.
5. Give constraints and indexes your own names.
6. Keep each migration small and about one thing.

See the history in the database

```
SELECT version, description, success, installed_on
FROM flyway_schema_history
ORDER BY installed_rank;
```

The schema after V5

Table	Column	Type and rules	Added in
products	`id`	BIGINT, identity, primary key	V1
	`name`	VARCHAR(255), NOT NULL, unique (`uk_products_name`)	V1, unique in V4
	`price`	NUMERIC(10, 2), nullable	V2
	`stock`	INTEGER, NOT NULL, default 0	V3
	`category_id`	BIGINT, nullable, foreign key to `categories.id`, indexed	V5
categories	`id`	BIGINT, identity, primary key	V5
	`name`	VARCHAR(100), NOT NULL, unique (`uk_categories_name`)	V5

Where things are in the project

Path	What it is
`src/main/resources/db/migration/`	The five migration files, V1 to V5.
`src/main/resources/application.properties`	Datasource placeholders and `ddl-auto=validate`.
`.env`	Database host, port, name, user and password. Not committed to git.
`src/main/java/.../product/`	Product entity, repository, service, controller and exception handler.
`src/main/java/.../category/`	Category entity, repository and read-only controller.